Assignment 2 - p26ai0009: Bagging and Boosting on Olivetti Faces

Mohammed Aflah
Roll No: P26AI0009
Repository: https://github.com/mohammedaflah/P26AI0009
Commit of code and results: 8bd3376b63f9bf48c5d3708c6068498a45b52170

Abstract

I compare a single decision tree, bagging and SAMME multiclass boosting on raw-pixel Olivetti faces (40 identities), with bagging and boosting written from scratch. Models are trained on clean images only and tested on clean faces and on faces with a centred 16×16 *block set to zero. Bias*^{2} *and variance are then measured over 30 within-identity resamples of the training set. Bagging reaches* 91.25% *clean test accuracy against* 61.25% *for the tuned tree and removes* 97% *of its variance, while its measured bias*^{2} *rises slightly. Boosting has the lowest variance but the lowest accuracy.*

1. Setup

Olivetti has 400 images of 64×64 pixels, 10 for each of 40 identities. The seed is every digit of the roll number, 260009, given to `numpy.random.default_rng`. Each identity's 10 images are split at random into 6 train, 2 validation and 2 test images $(240/80/80)$; the split is saved and reused everywhere. Features are raw flattened pixels, with no PCA or other reduction. The masked test set sets rows and columns 24–39 of each 64×64 test image to zero before flattening. All models train on clean images only. `DecisionTreeClassifier` is the only scikit-learn model used; bagging and SAMME are my own code. Ties in validation accuracy go to the earlier (shallower, smaller-leaf, fewer-learner) setting, and standard deviations use `np.std` with `ddof=0`.

2. Q1: Tree baseline and bagging

2.1. Single tree (Q1a)

Grid search over max_depth $\in \{2, 4, 8, 16, \text{None}\}$ and min_samples_leaf $\in \{1, 2, 4\}$ with clean validation accuracy only selected max_depth = None and min_samples_leaf = 1 (0.6750 validation, 0.6125 test accuracy). All three depth-`None` settings tie at 0.6750 validation accuracy, so the tie rule picks the first.

Table 1. Single-tree grid (selected rows). Leaf statistics use training counts. Depths 2, 4 and 8 gave identical results for all three leaf sizes.

depth	msl	Train	Val	Test	Leaves	Mean/leaf	1-sample
2	1	0.0750	0.0500	0.0750	3	80.00	0.000
4	1	0.1250	0.1000	0.1125	5	48.00	0.000
8	1	0.2250	0.2000	0.1875	9	26.67	0.000
16	1	0.4333	0.3625	0.3375	19	12.63	0.105
None	1	**1.0000**	**0.6750**	0.6125	46	5.22	0.130
None	2	0.9792	0.6750	0.6250	45	5.33	0.000
None	4	0.9750	0.6750	0.6500	40	6.00	0.000

What changes with depth. Training accuracy rises from 0.075 to 1.0, validation accuracy from 0.05 to 0.675, and the number of leaves from 3 to 46, while mean samples per leaf falls from 80.0 to 5.22. Capped trees underfit: depth 8 has only 9 leaves for 40 identities. The unrestricted tree fits the training set exactly, and 13.0% of its leaves hold a single training image. Its training accuracy exceeds validation accuracy by 32.5 points and test accuracy by 38.75 points, which indicates high variance.

2.2. Bagging (Q1b)

Each bootstrap sample draws $n = 240$ images with replacement and each tree uses the frozen setting. I fit one 200-tree ensemble and use its first B trees for size B, so the sizes are nested. The expected number of distinct images in a bootstrap sample is $n\big(1-(1-1/n)^n\big) = 151.89$ (63.3% of n; the limit is $1 - 1/e$). The measured averages in Table 2 range from 149.0 to 151.3 and approach the expectation as B grows, since they are means over more samples.

2.3. Diagnostics and choice of B (Q1c)

Validation accuracy is highest at $B = 200$ (0.9250), so $B = 200$ is the final size. $B = 50$ and $B = 100$ tie at 0.8875, so the lead of $B = 200$ is three validation images out of 80.

The three diagnostics do not tell the same story (Figure 1). OOB accuracy ($0.34 \rightarrow 0.77$) and vote margin

Table 2. Bagging versus B. Disagreement and margin use validation data; OOB is out-of-bag accuracy on the training set.

B	Train	Val	Test	Mean unique	OOB acc.	Pairwise disagr.	Vote margin
1	0.7500	0.3750	0.3750	149.00	0.3407	0.0000	1.0000
5	0.9667	0.6000	0.6125	149.00	0.3467	0.7775	0.2525
10	1.0000	0.7875	0.7500	149.90	0.4304	0.7997	0.2525
25	1.0000	0.8625	0.8750	150.08	0.5667	0.7838	0.2790
50	1.0000	0.8875	0.9000	150.82	0.6833	0.7857	0.2845
100	1.0000	0.8875	0.9250	151.05	0.7667	0.7789	0.3015
200	1.0000	**0.9250**	0.9125	151.29	0.7708	0.7772	0.3036

($0.25 \to 0.30$) keep rising with B, while pairwise disagreement stays near 0.78 for all $B \geq 5$. Disagreement describes how different individual trees are, which does not change as trees are added, so it cannot say when to stop. OOB accuracy trails validation accuracy at small B because a training image is out-of-bag for only about 37% of the trees, so its OOB vote comes from roughly $0.37B$ trees and is noisier.

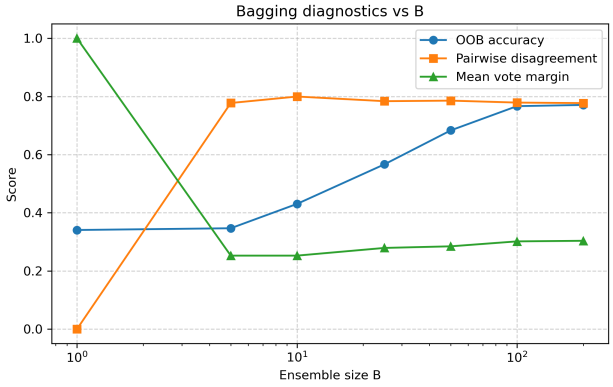


Figure 1. OOB accuracy, pairwise disagreement and mean vote margin versus B.

2.4. Clean versus masked, and case studies (Q1d)

Table 3. Single tree versus bagging ($B = 200$). Confidence is the mean winning vote fraction (tree: maximum leaf class probability).

	Accuracy		Confidence	
	clean	masked	clean	masked
Single tree	0.6125	0.4000	1.000	1.000
Bagging	0.9125	0.5750	0.394	0.239

Bagging is better on both sets, but its accuracy falls by 0.3375 under the mask against 0.2125 for the tree. The tree's confidence is 1.0 because the unrestricted tree has pure leaves even when it is wrong; the bagged winning vote falls from 0.394 to 0.239.

Table 4. Test faces (indices into the test set). Votes are out of 200; true class in bold.

case	true	tree	bag	top votes (class:count)
bag fixes tree (3)	1	6	1	**1**:75, 30:24, 12:17, 6:11
bag fixes tree (4)	2	12	2	**2**:77, 28:15, 25:12, 37:11
both wrong (0)	0	8	30	30:19, 4:17, 15:14, 1:12
both wrong (24)	12	16	14	14:22, **12**:17, 16:16, 17:13

What bagging can and cannot fix. In faces 3 and 4 the true class gets a clear plurality (75 and 77 of 200 votes) while the votes for the tree's wrong class are small, so the single tree's error came from the instability of one fitted tree, which is variance, and averaging removes it. In faces 0 and 24 the votes are spread over many classes and the true class is weak (class 0 is outside the top four; class 12 has 17 votes against 22 for the winner). Averaging trees trained on the same 240 images does not change that, so these errors are systematic.

3. Q2: SAMME boosting

3.1. Toy round (Q2a)

SAMME uses $\alpha_t = \log \frac{1-\epsilon_t}{\epsilon_t} + \log(K-1)$, $S_k(x) = \sum_t \alpha_t \mathbb{1}[h_t(x) = k]$ and confidence $q_k(x) = S_k(x) / \sum_t \alpha_t$. A weak learner with $\epsilon_t \geq 1 - 1/K$ is no better than random multiclass guessing; the code logs it, discards it and stops (keeping it only if it is the first learner), and a learner with $\epsilon_t = 0$ ends boosting after it is added. The toy example and the face model call the same update function. The toy set has three classes, ten samples and a depth-1 stump:

initial weights w_0	0.1 for all 10 samples
stump predictions	$[0, 0, 0, 0, 0, 0, 2, 2, 2, 2]$
misclassified	samples 4–6 (class 1, counting from 1)
weighted error ϵ_1	0.30
α_1	$\log(0.7/0.3) + \log 2 = 1.5404$
updated weights	0.0476 (correct), 0.2222 (wrong)

3.2. Selection and dynamics (Q2b)

Selection by clean validation accuracy gives depth 4 and 100 rounds (0.4875 validation, 1.0 training accuracy). This

Table 5. SAMME validation accuracy over weak-learner depth and rounds.

depth	$M = 10$	$M = 25$	$M = 50$	$M = 100$
1	0.0375	0.0375	0.1250	0.0750
2	0.1125	0.1000	0.1375	0.1625
3	0.1500	0.1500	0.4375	0.4625
4	0.2250	0.1875	0.4000	**0.4875**

is the corner of the prescribed grid and accuracy was still rising, so the optimum may lie outside it. Figure 2 shows the dynamics of the selected model.

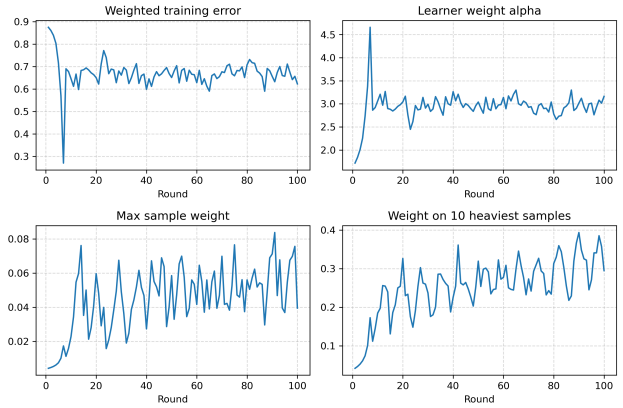


Figure 2. Depth 4, 100 rounds: weighted training error, learner weight α_t, maximum sample weight and weight on the 10 heaviest training images, per round.

3.3. Boosting versus bagging (Q2c)

Table 6. Equal numbers of learners L. Boosting uses depth-4 learners; bagging uses full-depth trees. Fit times are wall-clock seconds.

	Boosting				Bagging			
L	Train	Val	Test	Time	Train	Val	Test	Time
25	0.6042	0.1875	0.1750	7.1	1.0000	0.8625	0.8750	24.6
50	0.9375	0.4000	0.3000	14.2	1.0000	0.8875	0.9000	49.2
100	1.0000	0.4875	0.4000	28.4	1.0000	0.8875	0.9250	98.2

Bagging is more accurate at every size, while boosting is about $3.5\times$ cheaper to fit because its trees are shallow.

A heavily weighted face. Training image 177 (identity 29) reaches the largest weight of any sample, 0.0837 at round 91, about $20\times$ the uniform $1/240$. It is misclassified in 87 of the 100 rounds. In the first 20 rounds it is correct only at rounds 4, 9 and 16, and each time its weight drops sharply at the next round (for example $0.0061 \to 0.0008$ after round 4), then climbs again over the following rounds because later learners get it wrong. A misclassified sample's weight is multiplied by e^{α_t} before renormalising, so a face that shallow learners keep missing is pushed up and forces the next learner to attend to it again, which is why boosting keeps returning to it.

3.4. Clean versus masked for all three models (Q2d)

Table 7. Accuracy and mean winning vote (tree: leaf probability; bagging: vote fraction, $B = 200$; boosting: normalised weighted vote q_k).

	Accuracy		Mean winning vote	
	clean	masked	clean	masked
Single tree	0.6125	0.4000	1.000	1.000
Bagging	0.9125	0.5750	0.394	0.239
Boosting	0.4000	0.1875	0.115	0.137

The winning votes are on different scales: boosting spreads q_k over many classes, so 0.115 should not be compared directly with a bagging vote fraction. Under the mask boosting's winning probability rises slightly ($0.115 \to 0.137$) while its accuracy falls from 0.40 to 0.1875, so its confidence is not informative about accuracy here.

A face with different responses. For test face 6 (identity 3) under the mask, bagging predicts class 33 (37 of 200 votes, then 13:20, 35:16, 20:15) and boosting predicts the correct class 3 (probability 0.149, then class 20 at 0.083). The bagged trees disagree strongly on this face (pairwise disagreement 0.93), so the plurality of 37 is a small margin. The training images of identity 3 carry a mean boosting weight of 0.0046 against the uniform 0.0042, only a small increase, so sample weighting is a weak explanation for this single face. One face cannot establish why the two methods respond differently.

4. Q3: Bias and variance

I made 30 new training sets by drawing six faces with replacement, within each identity, from its six original training faces (about 4.0 distinct faces per identity on average, $6(1-(5/6)^6)$). The clean test set is fixed. I refit the selected tree (depth None, leaf 1), the final bagged model ($B = 200$) and the selected boosting model (depth 4, 100 rounds) with no re-tuning. The class vector $p_r(x_i)$ is the one-hot prediction for the tree, the vote fraction for bagging and the normalised weighted vote for boosting.

$$\text{Bias}^2 = \frac{1}{N}\sum_i \|\bar{p}(x_i) - e_i\|_2^2, \quad \text{Var} = \frac{1}{NR}\sum_{i,r} \|p_r(x_i) - \bar{p}(x_i)\|_2^2. \tag{1}$$

4.1. Selected models (Q3a)

Table 8. Measured over 30 refits. Accuracy is mean ± std (ddof 0) on the clean test set.

	Bias^2	Variance	Clean accuracy
Single tree	0.4177	0.6448	0.4688 ± 0.0488
Bagging	0.5036	0.0202	0.8025 ± 0.0538
Boosting	0.8896	0.0182	0.3004 ± 0.0774

Accuracy under resampling is below that of the frozen models (tree 0.469 vs 0.6125, bagging 0.8025 vs 0.9125) because each resample has only about four distinct faces per identity instead of six. Bagging removes 97% of the tree's variance ($0.645 \to 0.020$), but its bias^2 is higher ($0.418 \to 0.504$) although its accuracy is much higher. Bias^2 here is the squared distance between the mean class vector and the one-hot target. A tree's one-hot outputs can average close to the target, while bagged vote fractions are spread over several classes (mean winning vote about 0.4), which costs squared error even when the argmax is correct. Boosting's variance is low for a similar reason: its normalised votes are nearly flat (mean winning probability about 0.12), so they barely change between refits, and its bias^2 is the largest (0.89).

4.2. Depth sweep (Q3b)

Depths are 2, 8 and None with leaf size 1; the selected depth is None, so 8 is the middle value. Bagging uses 100 trees.

Table 9. Single tree versus 100-tree bagging at three depths (30 refits). Leaf statistics are for the tree fitted on the original training set; disagreement is among the 100 bagged trees on the test set.

	depth 2	depth 8	depth None
Tree bias^2 / var / acc	1.816 / 0.068 / 0.058	1.400 / 0.277 / 0.162	0.418 / 0.645 / 0.469
Bag-100 bias^2 / var / acc	0.934 / 0.010 / 0.148	0.812 / 0.016 / 0.423	0.504 / 0.024 / 0.790
Variance removed (%)	0.057 (84.5)	0.261 (94.2)	0.621 (96.2)
Accuracy gain	+0.089	+0.261	+0.322
Leaves / mean per leaf	3 / 80.0	9 / 26.7	46 / 5.2
Single-sample leaves	0.000	0.000	0.130
Test disagreement (bag)	0.955	0.914	0.782

With depth, single-tree bias^2 falls ($1.82 \to 1.40 \to 0.42$) and variance rises ($0.068 \to 0.277 \to 0.645$). Bagging removes 84.5%, 94.2% and 96.2% of the variance, or 0.057, 0.261 and 0.621 in absolute terms.

4.3. Is the deepest tree where bagging helps most? (Q3c)

In this experiment yes: the accuracy gain from bagging is $+0.089$, $+0.261$ and $+0.322$ at depths 2, 8 and None, but the measurements show why this need not hold in general. The gain tracks the variance available to remove (0.068, 0.277, 0.645), not depth itself. The shallow trees have 3 and 9 leaves and the highest bias^2 (1.82, 1.40), so most of their error is systematic and averaging cannot remove it (bagged depth-2 accuracy is only 0.148). The deep tree has 46 leaves with about five training images each and 13% single-image leaves, so its prediction depends on which few images were drawn, which is the variance bagging averages out. Disagreement among bagged trees is lowest at the deepest setting (0.782 against 0.955 at depth 2), so removing variance does not need high disagreement here. I measured only three depths, so I cannot say where between 8 and None the largest gain occurs.

4.4. Summary table (Q3d)

Table 10. Clean and masked test accuracy (mean over 30 refits), bias^2 and variance.

	Clean	Masked	Bias^2	Var
Single tree	0.4688	0.3138	0.4177	0.6448
Bagging	0.8025	0.5196	0.5036	0.0202
Boosting	0.3004	0.2746	0.8896	0.0182

Stability. Boosting (0.0182) and bagging (0.0202) have the lowest variance, against 0.645 for the tree. The spread of test accuracy across refits is similar for the tree and bagging (0.049, 0.054) and largest for boosting (0.077).

Sensitivity to the missing region. In the refit experiment bagging loses the most in absolute terms (0.283, against 0.155 for the tree and 0.026 for boosting), but boosting's small drop is a floor effect since it starts at 0.30. In the frozen comparison (Table 7) the drops are 0.3375 for bagging and 0.2125 for both the tree and boosting. Bagging still has the best masked accuracy in both experiments.

Lowest variance versus best accuracy. Boosting has the lowest variance, the lowest accuracy and the highest bias^2; bagging has slightly higher variance and far better accuracy. A model that is consistently wrong is stable, so variance alone does not rank models.

5. Reproducibility

All code, the saved split and resamples, configs, logs and figures are in the repository above. Running `python code/main.py` regenerates every table here; results are deterministic given the seed. Q3 takes about 54 minutes on an 8-core laptop.

References

[1] scikit-learn. `fetch_olivetti_faces` API and dataset documentation. https://scikit-learn.org/stable/

modules/generated/sklearn.datasets.fetch_olivetti_faces.html

[2] scikit-learn. DecisionTreeClassifier API. https://scikit-learn.org/stable/modules/generated/sklearn.tree.DecisionTreeClassifier.html